





20th edition hack.lu 2026 · lightning talk · **Rémi Séguy**

OpenTIDE

Opensource Threat-Informed Detection Engineering

Developed at the **CSOC of the European**

and open sourced since 2024 under the EUPL v1.2 EU public licence.

European Commission

opentide.org

github.com/opentidehq

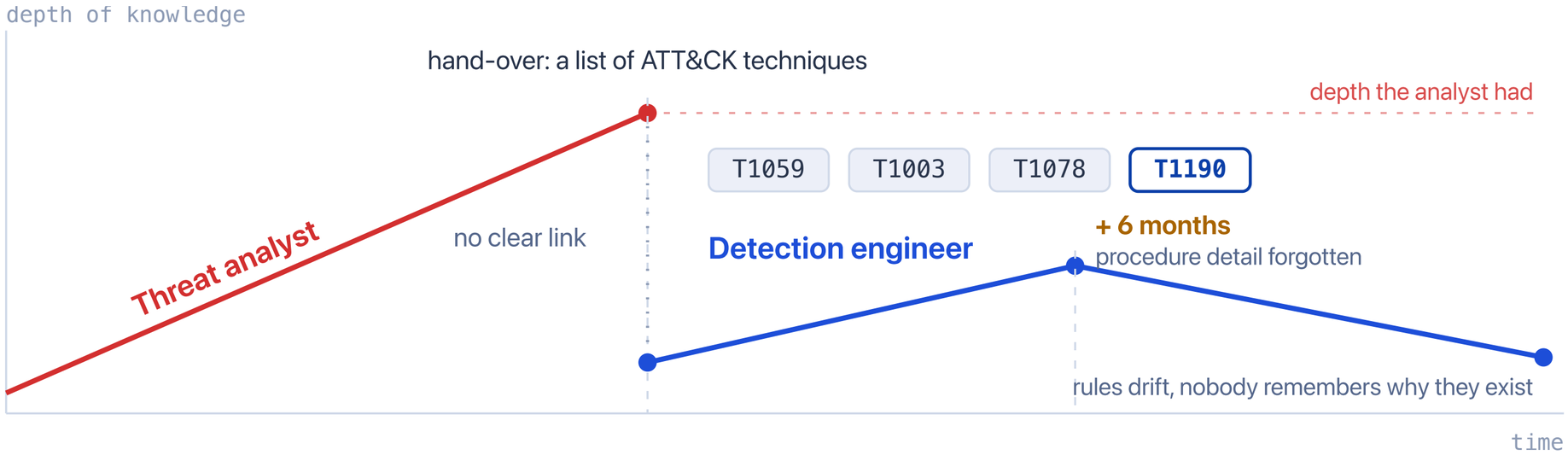
POSITIONING

opentide is a management and operational framework



WHY THIS EXISTS

ATT&CK technique IDs are a weak link



T1190

one technique id

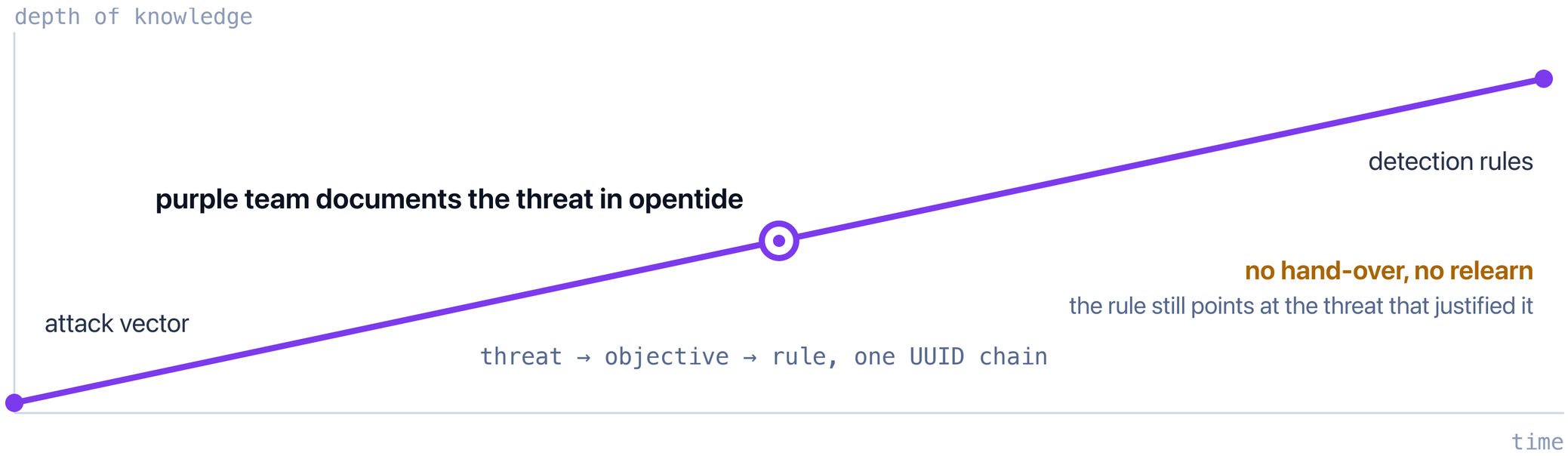
“Internet-facing application compromised.”

Which application? Which CVE? Which request, which log source, which field? You cannot write a query from this, and you cannot tell whether you already cover it — far too vague to drive detection engineering.

> A technique ID says *what category*, never *what the adversary actually did*. Six months on, the procedure behind the rule is gone and nothing in the repository can tell you whether the rule still matters.

WHY THIS EXISTS

With opentide: one curve, one chain



> Same people, same hours — one artefact. The threat is written down once, in a structured object, and the rule references it by UUID. Nothing to re-explain, nothing to forget.

THE MODEL

Turn TI in actionable objects down to rules

A rule without an objective is a query with no reason to exist. Make the agent earn each step.

threat

`threat::1.0`

What are we defending against?

`severity · impact · leverage`

`surface · mitre_attack`

`actors (vocabulary ids)`

covers

by uuid

objective

`objective::1.0`

What must we be able to see?

`priority · threats[uuid]`

`signals · methodology`

`entities · data requirements`

implements

by uuid

rule

`rule::1.1`

How does a platform see it?

`status · severity · techniques`

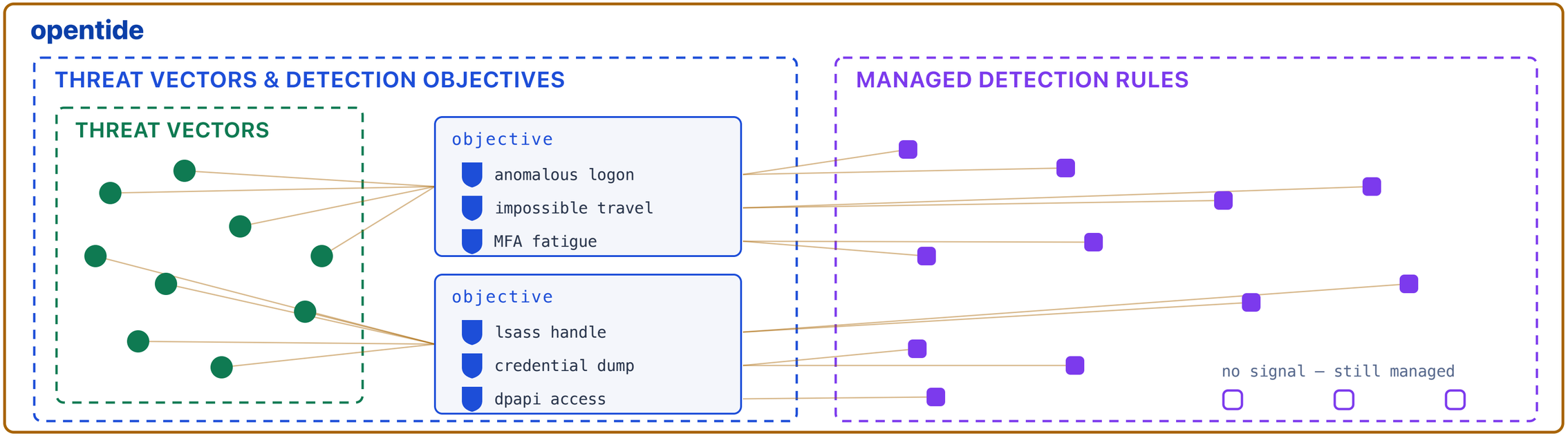
`detection_model[uuid]`

`configurations.sentinel.query`

> Coverage reads left to right; the YAML reference points back the other way, in blue. The UUID is the identity — rename and move files freely.

USE CASES

From a threat library to managed rules



> An objective decomposes into **signals** — the evidence that would satisfy it — and a rule's `detection_model` names the signal it implements, not the whole objective. That field is optional: the hollow rules validate and deploy like any other, they just cannot tell you why they exist.

OPERATIONS

The rules implement the Threat Detection

LEVEL 1 Objects · the threat-informed chain

Threats — what we defend against → **Objectives** — what we must see → **Signals** — the evidence → **Rules** — how a platform sees it

LEVEL 2 Rule development status · status decides what deploy does

DESIGN	DEVELOPMENT	STAGING	ACCEPTANCE	PRODUCTION	IMPROVING	DISABLED	REMOVED
inert	preview	preview	preview	release	loops back	disablement	deletion

LEVEL 3 Platforms · eight in the wheel, two on the roadmap

Microsoft Sentinel sentinel deploy · KQL check	**Defender for Endpoint** defender_for_endpoint deploy · KQL check	**Splunk ES** splunk deploy · SPL check	**SentinelOne** sentinel_one deploy · S1QL check	**Carbon Black Cloud** carbon_black_cloud deploy · Lucene check
Elastic Security elastic deploy · ES|QL · EQL check	**CrowdStrike Falcon** crowdstrike deploy · no query check	**HarfangLab** harfanglab deploy · no query check	**Sysdig Secure · Falco** RFC 0004 draft — not shipped	**Graylog** community ask not planned yet

03 · why we re-released it

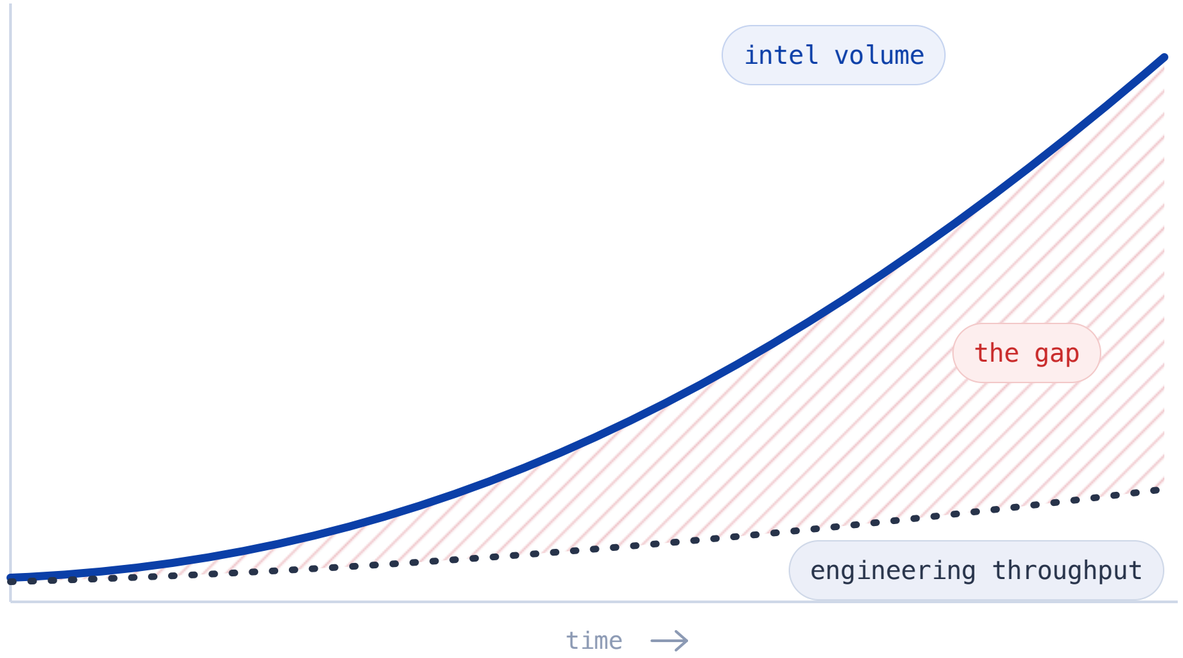
Why we re-released it

Years in production as a git submodule. Now a versioned CLI on PyPI, with the ergonomics humans and agents both need.

WHY IT MATTERS

Intelligence outruns detection engineering.

The bottleneck was never the strategy. It was the capacity to execute it: read, model, write, tune, deploy, repeat.



1,000s

intel items a month: advisories, CVEs, campaigns

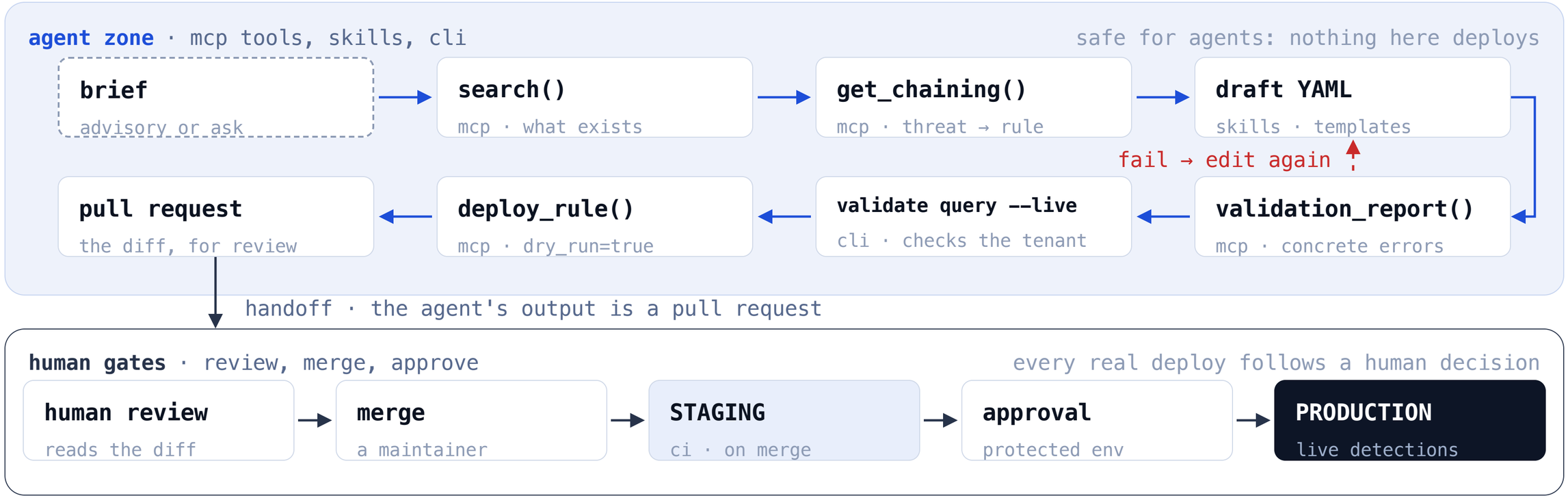
hours

per TTP to read, model, write, tune and deploy

tens

production-ready detection rules a month

The loop: agents draft, the engine checks, humans promote.



GENERATED ARTEFACTS

Write the objects once, get the rest for free

Documentation, coverage exports and sharing are not side projects — they fall out of the validated objects, so they cannot drift from what you actually run.

DOCS

A wiki that cannot go stale

```
opentide generate docs
opentide generate docs --changed
```

- A markdown page per threat, objective and rule, with Mermaid chain diagrams
- Backlinked both ways: rule → signal → objective → threat, and back down
- Flavours for GitHub, GitLab and Azure DevOps

EXPORTS

Coverage you can show a board

```
opentide generate exports navigator
opentide generate explorer
```

- ATT&CK Navigator layer 4.5 — covered, a real gap, a rule with no threat behind it
- Graph explorer: the whole catalogue as a static site, with a prebuilt search index and no backend to patch

SHARE

Publish objects to MISP

```
opentide share preview
opentide share push --changed
```

- One MISP Event per object — preview first, which makes no request
- `max_tlp` is a hard ceiling per destination, so a TLP:AMBER object cannot leave
- Sharing is not deployment: a destination is a MISP instance, not a platform

> Nobody writes these by hand, so nobody can forget to update them. On the Navigator layer the red squares are your real gaps — and the blue ones are the rules nobody can justify yet.

WRAP UP

Where to go next

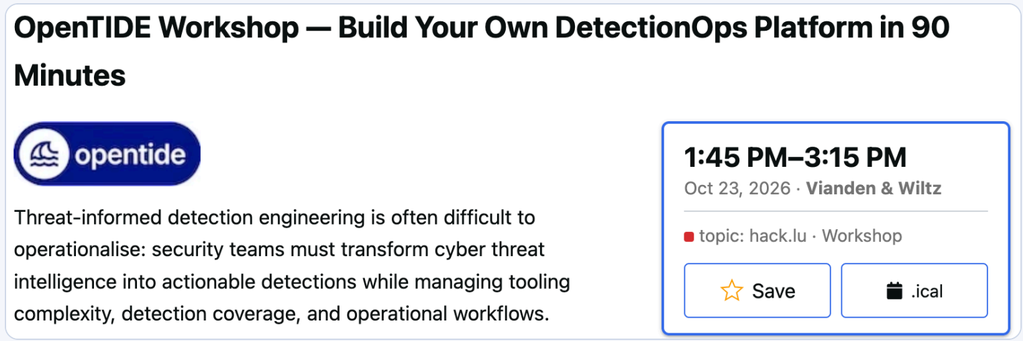
DOCS, CODE, LIBRARY

opentide.org

github.com/opentidehq
pypi.org/project/opentide
opentidehq/library

Docs, the engine, the specifications, and a library of objects you can reuse.

WORKSHOP · HACK.LU 2026



Come to the workshop with your agent!

Thank you!